



THE EDUCATOR · CPCS 351

Topic 02

Software Quality

A complete, study-ready edition of the live Topic 02 content -
with original repository visuals and one consistent Golden Master design system.

SQA & Evidence	Cost of Quality	Verification & Validation
Static & Dynamic V&V	Reviews & Inspections	Metrics & Attributes

9 lessons · complete lecture content · activities · self-feedback · visual anchors

Golden Master · v1.0

Contents

Nine lessons forming one quality-engineering story: from SQA and cost of quality to verification, validation, reviews, measurements, quality attributes, and long-term engineering trade-offs.

Software Quality & SQA	3
Cost of Quality	5
Verification vs. Validation	7
Static & Dynamic V&V; Across the Life Cycle	8
Inspections, Walkthroughs & Peer Review	10
Quality Measurements & Metrics	13
Software Quality Attributes	16
Short-Term vs. Long-Term Quality	18
Software Engineering vs. Computer Science	19

Golden Master rule

This PDF preserves the instructional content of the live Topic 02 pages. Original image-based visual anchors are rendered from the repository assets; the remaining visual structures are reproduced in the same print design system used for Topic 01.

Software Quality & SQA

1

of 9 lessons

Build a complete mental model of software quality assurance before studying its techniques.

Hook

Can software pass every test you wrote and still be a low-quality product?

Yes. Quality is broader than “no observed bugs”: it includes conformance to requirements and standards, fitness for real customer needs, and qualities such as reliability and maintainability.

Learning Outcome · CLO #2

Define software quality and explain the role of quality assurance activities in the software process, then distinguish verification, validation, reviews, testing, metrics, and quality attributes in context.

PRACTICE LAB · TOPIC 02

Mission 02 - Build the Quality Case

Apply the complete quality story to a mobile banking release through nine diagnostic engineering stages covering quality cost, V&V, evidence, metrics, quality attributes, and quality over time.

TOPIC 02 · MASTER QUALITY MAP

Quality is built through a chain of evidence

Quality Goals	Cost of Quality	Verification
Validation & Testing	Reviews & Inspections	Metrics
Quality Attributes		

The chapter asks one continuous question: How do we create credible evidence that the software is correct, useful and sustainable? We invest in prevention and appraisal, inspect artifacts before execution, execute software when behavior must be observed, measure what can be quantified, and judge quality against attributes that matter now and over time.

MUST KNOW

What Is Software Quality Assurance?

Software Quality Assurance (SQA) is a set of activities used to ensure that the software product and/or process conforms to established requirements and quality standards.

The lecture organizes SQA around four recurring mechanisms:

Are we building the product right?	Are we building the right product?
Examine artifacts without waiting for execution	Execute software to uncover errors

QUALITY
Product + Process

Think of SQA as a shield built throughout the life cycle-not a final gate added after coding.

EVIDENCE MODEL

Do not collapse these terms into one bucket

Requirements, design and code can be examined without executing the software. Reviews and inspections can therefore find defects early.	Testing requires execution and compares observed behavior with expected behavior or needs.
Does the work product conform to its specification, standards or preceding artifacts?	Does the resulting product/model satisfy the intended real-world need?

The Story of This Topic

We begin with why quality costs money, then learn the two central questions of verification and validation. From there we examine static and dynamic techniques, move into reviews and inspections, quantify quality through metrics, and finally ask what “high quality” actually means now and over the product’s lifetime.

Common Misconception

“SQA = testing.” Testing is one SQA activity. Reviews, verification, validation, standards, measurements, and process-oriented quality work all contribute evidence before and beyond execution.

Apply it · Online Examination System

A requirement says “After time expires, answers shall be submitted automatically.” A reviewer checks the design against this requirement; later a tester runs an exam until the timer expires; students then report whether automatic submission actually supports the intended examination workflow. Identify the kinds of evidence.

Reveal reasoning: design checking is static verification; executing the timer scenario is dynamic testing and can provide verification evidence against the stated requirement; evaluating whether the behavior serves the real examination need is validation-oriented evidence. The same project needs several kinds of quality evidence.

Cost of Quality

2

of 9 lessons

Understand why quality spending is an economic engineering decision, not optional overhead.

Hook

Would you rather pay to prevent a defect today-or pay for the same defect after thousands of customers depend on it?

Quality has a cost, but failure has a cost too. Software engineering balances both.

Learning Focus

Explain the two major components of cost of quality and why defect-removal cost rises as defects escape into later life-cycle stages.

OFFICIAL LECTURE CONTENT

Two Sides of the Cost of Quality

Prevention / Assurance Cost

- Implementing an SQA framework - once.
- Performing SQA activities - ongoing.
- Monitoring and improving the framework - ongoing.
- Equipment, machines, and tools.

Failure Cost

- Analyzing and removing failures.
- Lost productivity for developers and customers.
- Liability, lawsuits, and higher insurance premiums.
- Damage to the company's reputation.

The lecture's key trade-off is simple: spending nothing on prevention does not make quality free-it shifts spending toward failure.

VISUAL ANCHOR

The Later a Defect Escapes, the More Expensive It Becomes

Stage	Relative correction cost
Requirements	■ ~1×
Design	■■ ~5×
Coding	■■■ ~10×
Development Test	■■■■■ ~50×
System Test	■■■■■■■ ~100×
Field Operation	■■■■■■■■■■ ~1000×

The original lecture illustrates relative correction cost climbing dramatically—from the order of 1 in early stages toward values as high as 1000 in field operation. The exact multiplier is less important than the engineering lesson: find problems early.

Decide Where to Spend

A team says requirements reviews are “too expensive” and wants to save all quality effort for system testing. What cost are they ignoring?

Self-Feedback: They are ignoring defect propagation. A requirement defect can contaminate design, code, tests, documentation, deployment, and customer operations. Prevention cost is visible early; failure cost is often much larger and arrives later.

Verification vs. Validation

3

of 9 lessons

Separate conformance to specification from correspondence to real customer needs.

Hook

A system can perfectly implement the wrong requirement. Is that success?

This is why software quality asks two different questions: conformance and real-world fitness.

Learning Focus

Distinguish verification from validation and identify which question each technique answers.

Verification checks whether an implementation conforms to its specification. A lower-level abstraction implements a higher-level abstraction. For example, design verification checks whether software design satisfies the requirements specification.

Validation checks correspondence between a model/product and the real world or real customer need. The lecture describes it as a process that seeks to refute a model. Functional testing is given as a validation technique.

Use the Abstraction Ladder



Verification asks whether each lower-level representation correctly implements the higher-level one. Validation asks whether our representations and final system correspond to what is actually needed.

Common Misconception

“Verification is developer testing and validation is customer testing.” The lecture’s distinction is conceptual: conformance to specification versus correspondence to real needs.

Diagnose the Failure

A hospital system exactly implements “send reminders 24 hours before the visit,” but users actually need 48 hours because of transport constraints. Verification or validation problem?

Self-Feedback: The implementation may verify correctly against the written requirement, yet the requirement fails validation against the real-world need.

Static & Dynamic V&V Across the Life Cycle

4

of 9 lessons

See how quality evidence follows artifacts from requirements through operation.

Hook

How can you find a serious defect before the software even runs?

By examining static representations: requirements, designs, code, test data, and other artifacts.

Learning Focus

Explain static versus dynamic V&V and map appropriate quality activities across the software life cycle.

Conceptual Anchor · Verification vs. Validation

Two different quality questions

Verification checks whether a work product or implementation conforms to its specification, standards, or preceding higher-level artifact. Examples include reviewing an SRS, checking a design against requirements, or inspecting code against the design.

Validation checks whether the product or model corresponds to the real-world need and intended customer use. It asks whether the result is actually useful and appropriate for its intended purpose.

Important: verification and validation describe questions. Static and dynamic describe ways of gathering evidence. Do not treat “verification = static” and “validation = dynamic” as equivalent definitions.

Static

Review, walkthrough, inspection, peer review, and desk checking analyze a static representation to detect problems without executing software.

Dynamic

Testing executes the product and observes its behavior to check results and uncover failures.

LIFE-CYCLE EVIDENCE MAP

Quality Evidence Follows the Life Cycle

Life-cycle stage	Evidence / quality activity
Requirements Analysis	SRS review Verification

Software Design	Design review Verification
Coding & Unit Testing	Code inspection Unit testing
Integration & Integration Testing	Integration testing
Acceptance Testing	Validation & testing
Maintenance	Review changes Regression evidence

The lecture emphasizes three simultaneous concerns: development activities must be carried out correctly; artifacts such as the SRS, design, and code must be checked; and integrated software must execute and produce expected results.

Map the Technique

You have only a requirements document-no code exists yet. Which quality activities are still possible?

Self-Feedback: Static techniques are already possible: review, inspection, walkthrough, or peer review of the requirements. Quality assurance begins before executable software exists.

Inspections, Walkthroughs & Peer Review

5

of 9 lessons

Compare the major verification and validation techniques by purpose, execution, participants, structure and the artifacts they can examine.

Hook

Five techniques can all contribute to quality-but they do not look, feel, or operate the same way.

Use the visual anchors below to remember the difference before memorizing definitions.

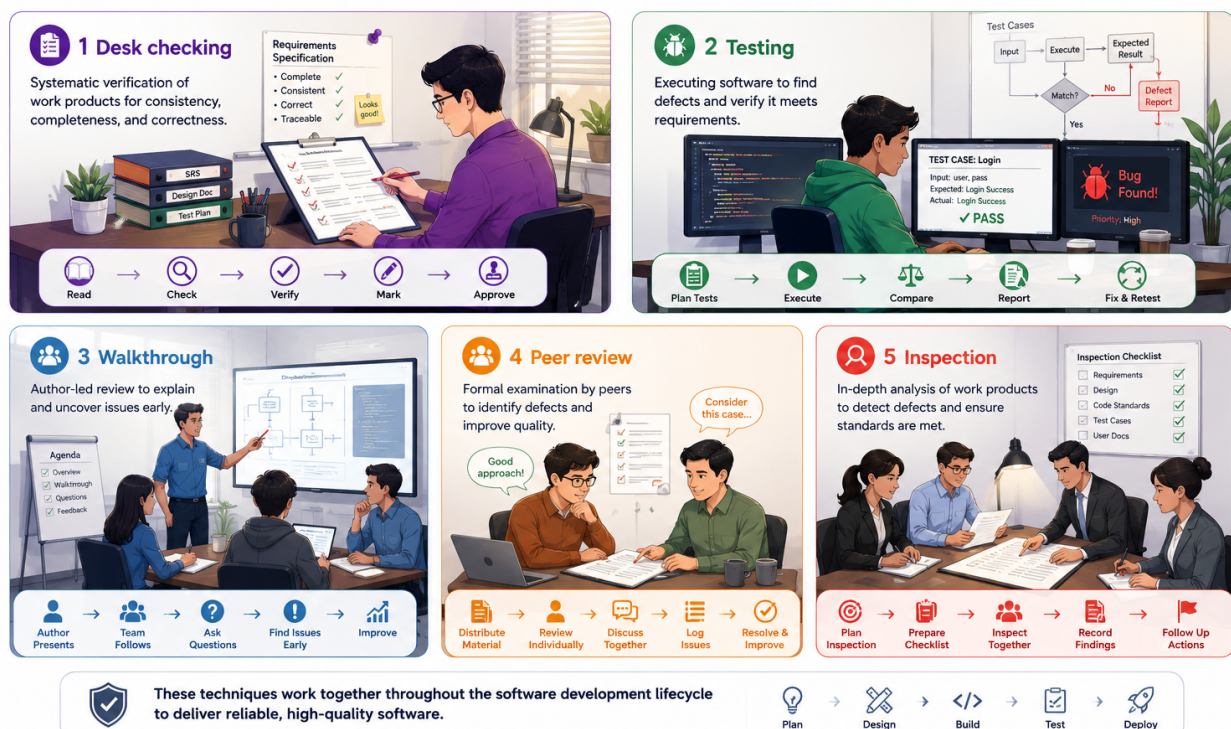
Learning Focus

Distinguish desk checking, testing, walkthrough, peer review, and inspection by execution, participants, structure, purpose and applicable artifacts.

VISUAL ANCHOR · SOFTWARE VERIFICATION AND VALIDATION TECHNIQUES

Software Verification and Validation Techniques

Ensuring the right product is built right



The five techniques in one large study visual: use the scene for each technique as its memory anchor, then use the detailed explanations below to distinguish purpose and procedure.

PURPOSE SHARED BY PEER-REVIEW PROCESSES

What Reviews Are Trying to Achieve

- Determine whether the work product satisfies its specifications.

- Detect deviations from standards and expected practice.
- Identify defects, inconsistencies and suspicious anomalies.
- Provide constructive suggestions for improvement where appropriate.

Important: review and inspection are not limited to source code. Static representations can be examined before executable software exists.

Requirements / SRS	Design	Source Code
Test Data	Other Static Artifacts	

Five Techniques - Fast Comparison

Static · Individual Desk Checking One person reads the code or artifact alone. It is informal and less effective than walkthrough or inspection.	Dynamic · Execute Testing Execute the software and observe behavior. Testing attempts to uncover program errors.	Static · Guided Walkthrough The developer manually walks through the work, often using simple test data, while participants ask questions.
Static · Peers Peer Review Peers evaluate independently, answer review questions, then discuss findings with the developer.	Static · Checklist Inspection A structured defect-detection examination using criteria, common-error checklists, standards and consistency rules.	

Static vs. Dynamic: The Fast Separation

Desk checking, walkthrough, peer review, inspection. Analyze a representation without executing the product.	Testing. Execute the product and observe what it does.
--	--

Program Inspection: What Counts as a Defect?

Examples include logical errors, suspicious anomalies such as an uninitialized variable, and non-compliance with standards. The inspector's goal is detection; correction is a later responsibility. Inspection does not require execution, can be used before implementation, and can examine requirements, design, code, test data and other system representations.

Walkthrough vs. Inspection

Walkthrough

Uses simple test data, manual simulation, step-by-step reading aloud, and discussion that stimulates questions and doubt.

Inspection

Uses a defined list of criteria: common errors, standards, and consistency rules. The team systematically examines the product against those criteria.

Peer Review Procedure

The reviewers may have one to two weeks to evaluate the product. Results can vary with reviewer knowledge, experience, background, and criticality.

1	Present a product overview and assign material to reviewers.
2	Reviewers evaluate independently and answer the review questions.
3	At the review meeting, reviewers present comments; the developer answers questions and clarifies doubts.
4	The developer fixes problems and produces a summary of solutions.
5	Reviewers check the changes; a second meeting may be needed.

WORKED EXAMPLE

One Requirement, Three Review Styles

Requirement: “The system shall lock an account after five failed logins.” A desk check is one reviewer reading it alone. In a walkthrough, the author guides peers through login scenarios. In an inspection, reviewers systematically check the requirement against defined criteria such as ambiguity, consistency and standards. Later, testing executes failed logins dynamically.

Choose the Technique

Your goal is consistent detection of known coding-standard violations across a team. Walkthrough or inspection?

Self-Feedback: Inspection is the stronger fit because it explicitly uses criteria/checklists for common errors, standards, and consistency rules. A walkthrough is better suited to manual simulation and discussion around the developer’s work.

Quality Measurements & Metrics

6

of 9 lessons

Turn quality claims into quantitative evidence using measurements and metrics across requirements, design, implementation and object-oriented software.

Hook

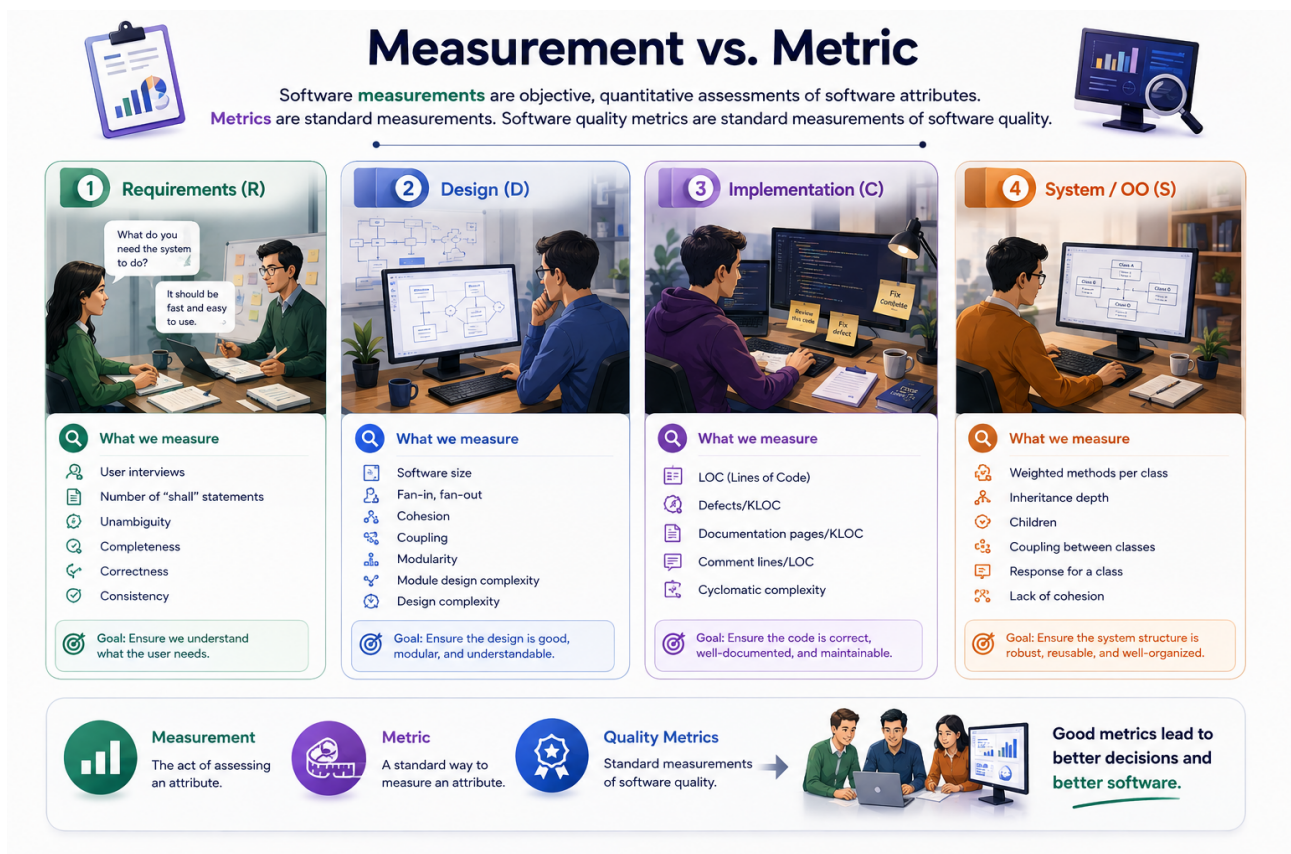
If a manager says “quality improved,” what evidence would make that statement more than an opinion?

Measurements turn vague claims into evidence-but only if we understand what each metric actually measures.

Learning Focus

Define measurement and metric, identify the complete R/D/C/S metric categories from the lecture, and interpret representative requirements, design, implementation, and OO metrics.

VISUAL ANCHOR · MEASUREMENT VS. METRIC



Use this visual to anchor the distinction between software measurements and standardized quality metrics.

Measurement vs. Metric

Software measurements are objective, quantitative assessments of software attributes.

Metrics are standard measurements. Software quality metrics are standard measurements of

COMPLETE LECTURE TAXONOMY

Four Families of Software Quality Metrics

R · Requirements Unambiguity, Completeness, Correctness, Consistency.	D · Design Fan-In, Fan-Out, Cohesion, Coupling, Modularity, Module Design Complexity, Design Complexity.
C · Implementation Defects/KLOC, documentation/KLOC, comment lines/LOC, cyclomatic complexity.	S · System / OO Weighted methods/class, inheritance depth, number of children, class coupling, response for a class, lack of cohesion in methods.

Requirements Metrics - Complete List

- Unambiguity: degree to which requirements have a single clear interpretation.
- Completeness: degree to which required information and behavior are specified.
- Correctness: degree to which requirements accurately represent what is required.
- Consistency: degree to which requirements do not conflict with one another.



The rewritten statement is an explanatory example: it shows how a vague quality claim can become easier to verify and measure.

Design Metrics - Definitions from the Lecture

- Fan-In: number of modules that call a given module.
- Fan-Out: number of modules called by a given module.
- Cohesion: relevance of functions in a module to its core functionality.
- Coupling: data dependencies among modules.
- Modularity: high cohesion and low coupling lead to high modularity.
- Module Design Complexity: integration tests required to integrate a module with subordinate modules.
- Design Complexity: number of subtrees in structure charts with module M as root.



Module A illustrates the direction associated with stronger modularity: higher cohesion and lower coupling.

Implementation Metrics - Complete List

- Defects/KLOC: defects normalized by thousand lines of code.
- Pages of documentation/KLOC: documentation volume normalized by code size.
- Comment lines/LOC: comment density relative to source code.
- Cyclomatic complexity: number of independent control-flow paths in a function (McCabe, 1979).

Example: two modules may contain the same LOC but differ greatly in cyclomatic complexity; size alone therefore does not describe control-flow complexity.

Object-Oriented / System Metrics - Complete List

- Weighted methods per class.
- Depth of inheritance tree.
- Number of children.
- Coupling between object classes.
- Response for a class.
- Lack of cohesion in methods.

Example: a class with many unrelated methods may show weak cohesion even when it compiles and passes basic functional tests. Metrics provide a signal for investigation; they do not replace engineering judgment.

Common Misconception

"A number is automatically a quality metric." A metric is useful only when the measured attribute, definition, context, and interpretation are clear.

Interpret, Don't Just Count

Two modules have equal size. A has high cohesion and low coupling; B has low cohesion and high coupling. Which is more modular?

Self-Feedback: Module A. The lecture explicitly links high cohesion + low coupling with high modularity.

Software Quality Attributes

7

of 9 lessons

Understand high quality as a multidimensional profile, not a single score.

Hook

What does “high-quality software” mean if two users value different things?

A fast system can be hard to maintain; a reliable system can still be difficult to use.

Learning Focus

Define the five quality attributes emphasized in the lecture and diagnose which attribute is at stake in a scenario.

Usability Learn quickly and get the job done easily	Efficiency Avoid wasting CPU time and memory	Reliability Do what is required without failing
Maintainability Be easy to change	Reusability Reuse parts in other projects	

Quality Is a Profile, Not a Single Score

The lecture’s core vocabulary is Usability, Efficiency, Reliability, Maintainability, and Reusability. Their relative importance depends on context.

VISUAL SCENARIO MAP

See Each Attribute in a Real Situation

Usability · RegistrationA new student can find a course, understand prerequisites and register without training or unnecessary steps.	Efficiency · SearchTwo systems return the same correct result, but one consumes far less CPU time and memory.	Reliability · PaymentA payment service repeatedly performs the required transaction correctly without failing under expected conditions.
Maintainability · Policy ChangeA new registration rule can be added by changing a focused module rather than rewriting unrelated parts.	Reusability · AuthenticationA well-designed authentication component can be reused by registration, advising and alumni applications.	

These are explanatory examples that apply the lecture definitions; they are not additional quality attributes.

Diagnose the Attribute

A system produces correct results but consumes so much memory that ordinary laptops freeze. Which quality attribute is weakest?

Self-Feedback: Efficiency. Reliability may still be acceptable if results are correct, but inefficient resource use is explicitly the issue.

Common Misconception

“If it works, it is high quality.” Functional correctness matters, but usability, efficiency, reliability, maintainability, and reusability describe different dimensions.

Short-Term vs. Long-Term Quality

8

of 9 lessons

Judge software quality across today's needs and tomorrow's change and scale.

Hook

A system satisfies today's users perfectly. Does that guarantee it is high quality?

Not if tomorrow's data volume, features, or business needs make it expensive or impossible to evolve.

Learning Focus

Contrast short-term and long-term quality and explain why maintainability, future needs, and scalability matter.

Short-Term Quality

- Does the software meet the customer's immediate needs?
- Is it sufficiently efficient for the volume of data we have today?

Long-Term Quality

- Maintainability.
- Ability to support the customer's future needs.
- Scalability: can it handle larger volumes of data?

The Engineering Tension

"Good enough today" may be rational-but only when future change and growth risks are consciously considered. Overengineering everything is wasteful; ignoring likely evolution creates expensive technical debt. This connects directly to Topic 01's PQCT trade-offs.

Look Beyond Today

A registration system handles 2,000 students perfectly but cannot scale to 50,000 without a rewrite. Is the current system low quality?

Self-Feedback: Short-term quality may be acceptable, but long-term scalability is weak. The judgment depends on whether growth to 50,000 is a realistic future requirement.

Software Engineering vs. Computer Science

9

of 9 lessons

Connect quality engineering to the broader engineering trade-offs introduced in Topic 01.

Hook

Why would a software engineer knowingly choose a solution that is not mathematically optimal?

Because real engineering optimizes under cost, schedule, risk, people, maintainability, and changing requirements-not technical elegance alone.

Learning Focus

Contrast the lecture's framing of computer science and software engineering, then integrate the major ideas of Topic 02.

Computer Science

- Pursue optimal solutions.
- Cost is not the primary consideration.
- Programming in the small.
- Technical issues.
- Dealing with tame problems.

Software Engineering

- "Good enough" can be enough.
- Cost matters - PQCT.
- Programming in the large.
- All issues and aspects.
- Dealing with wicked problems.
- Builds on computer science and other disciplines.

TOPIC 02 REVIEW

Key Takeaways

- SQA ensures product/process conformance to requirements and quality standards.
- Quality cost includes prevention/assurance and failure costs; defects become more expensive when discovered late.
- Verification: implementation/work product vs. specification. Validation: product/model vs. real need.
- Static techniques examine artifacts without execution; dynamic testing executes software.
- Inspection, walkthrough, desk checking, and peer review differ in rigor, process, and purpose.
- Metrics make attributes measurable, but require interpretation and context.
- Quality attributes include usability, efficiency, reliability, maintainability, and reusability.
- Quality has short-term and long-term dimensions, including scalability and future change.

WHOLE-TOPIC TRANSFER CHALLENGE

Quality-engineer a Mobile Banking Release

A new release adds instant international transfers. Use the entire topic rather than one technique.

What prevention/appraisal work is worth paying for before release, and what failure costs could appear afterward?	What evidence shows the implementation follows specifications, and what evidence shows customers can safely accomplish the intended task?
What can be reviewed without execution, and what behavior must be executed?	Which artifacts deserve peer review or formal inspection before deployment?
What measurable indicators could support decisions about defects, complexity, reliability or maintainability?	Which qualities matter immediately, and which design choices determine whether the feature remains safe and maintainable as regulations and traffic change?

Reveal a model reasoning path

Invest early in requirement/security reviews, design inspection and test planning because post-release financial failures are expensive. Verify transfer limits, authorization rules and audit behavior against specifications; validate that the workflow meets real customer and regulatory needs. Review requirements/design/code statically and execute transaction, failure, load and recovery scenarios dynamically. Use defect and complexity/reliability measures as evidence rather than goals by themselves. Judge immediate correctness/usability together with long-term maintainability, scalability and adaptability.

UNDERSTANDING CHECK

Can You Explain the Whole Quality Story?

Why can early quality activities reduce total project cost?	Give one verification and one validation example.
Name one requirements, design, implementation, and OO metric.	Give a case where short-term quality is good but long-term quality is weak.

Self-Feedback: A strong answer uses lecture vocabulary accurately and connects quality activities to life-cycle timing, cost, measurable evidence, and real user needs.

EXAM LENS · CRITICAL DISTINCTIONS

Do not confuse the question with the technique

These describe different quality questions; do not reduce them to “static vs. dynamic.” Verification can use static or dynamic evidence depending on what is being checked.	A review examines an artifact; testing executes software. Both can uncover defects, but through different mechanisms.
A metric is evidence about an attribute, not the attribute itself and not proof of quality in isolation.	A release can work today yet accumulate design choices that make tomorrow's change, scale or maintenance expensive.

NEXT TOPIC

Topic 03

System Engineering

Topic 03 moves from software quality to the larger system context: System Engineering. Carry one idea forward—software quality is influenced by the system, stakeholders, constraints, and environment around it.

Carry forward the quality-engineering mindset: software quality is shaped by the wider system, its stakeholders, constraints, and operating environment.